GridMind — GEAP Track 3 Compliance Audit

The Fortified Enterprise Fleet · project `gridmindai-507000` · audited against the seven recommended Gemini Enterprise Agent Platform capabilities

Verdict

Four of the seven GEAP capabilities are genuinely implemented, one is partial, and two are absent. Where a capability exists it was built directly on Google Cloud primitives rather than by adopting the GEAP managed product — the behaviour is real and verifiable, but it is our implementation, not the branded service. This document states which is which, because the difference is exactly what a technical reviewer will test.

The hackathon's **hard requirements are fully met**: Gemini 3.5 via Vertex AI, the Google GenAI SDK as the agent framework, and Cloud Run plus Firestore as infrastructure. The seven GEAP tools are *recommended*, not mandatory.

Scorecard

GEAP capability	Status	What exists in GridMind today
Agent Registry	**NOT BUILT**	No discovery or versioning catalogue. Artifact Registry stores the container image; that is image storage, not agent discovery. Nothing lets an organisation find an agent, read its contract, or pin a version.
Agent Runtime	**PARTIAL**	Six Cloud Run services with per-agent identity, scale-to-zero and bounded max-instances. Execution is synchronous request/response, so a 60–90 s negotiation blocks the caller. No durable long-running or async job state.
Memory Bank	**BUILT (own implementation)**	Firestore `memory_bank` collection. The orchestrator recalls a precedent by conflict type before reconciling, and writes a new precedent after any multi-round resolution. Confirmed recalling a precedent written by an earlier session.
Agent Identity	**BUILT (strongest area)**	Seven service accounts, three custom least-privilege roles, IAM Conditions pinning each agent to one Firestore database, a Cloud Run invoker chain, and VPC internal ingress. 21 automated checks pass.
Agent Gateway	**BUILT (own implementation)**	FastAPI service performing caller identity verification, routing-table policy, audience-scoped token minting, and allow/deny audit logging. The orchestrator cannot reach a specialist except through it.
Model Armor	**NOT BUILT**	Descoped during planning. No screening for prompt injection, tool poisoning or PII leakage on inter-agent traffic. **The API is available in this project** and the gateway is the natural integration seam.
Agent Observability	**BUILT**	Structured JSON to Cloud Logging, a correlation ID threaded through every agent in a decision, and the complete per-round reasoning chain persisted to `negotiation_log`.

Read the scorecard as 4 built / 1 partial / 2 missing. The two gaps are not cosmetic: **Agent Registry is the single weakest point against this track's stated focus**, which opens with the words "corporate agent discovery".

Against the five stated focus areas

The track description names five things a submission should show. This is how GridMind maps onto them.

Focus area	Rating	Evidence
Corporate agent discovery	**WEAK**	There is no registry. An organisation cannot discover the agents, inspect their input/output contracts, see which data domain each is scoped to, or pin a version. This is the clearest gap in the submission.
Multi-agent orchestration at scale	**STRONG**	Five agents across six services. Round 1 runs the four specialists concurrently and independently; the orchestrator then tests whether four verdicts describe ONE physically deployable plan — not whether they agree. Conflicts open bounded re-prompt rounds; unresolved conflicts escalate to a human rather than being silently decided.
Long-term state persistence	**STRONG**	Memory Bank precedents plus a complete negotiation log survive across sessions and are re-consulted by later decisions. State is a queryable audit record, not a conversation buffer.
Runtime observability	**STRONG**	Every model call, Firestore read, guardrail violation, retry and gateway decision emits a structured event under one correlation ID. A whole multi-agent decision is reconstructable from Cloud Logging with a single filter.
Security posture enforcement	**STRONGEST**	Four independent layers — identity, network, data, audit — each demonstrated by a real denial rather than described. Weakened only by the absence of Model Armor at the content layer.

What the security posture actually enforces

The load-bearing technical finding: **Firestore Security Rules are not evaluated for server-side Admin SDK access.** They apply only to client SDK and Firebase Auth traffic. A design that scoped agents with per-collection Security Rules would therefore have enforced nothing at all, while appearing correct in a diagram.

GridMind instead uses one Firestore database per domain, with each service account bound through an IAM Condition on `resource.name` — which *is* enforced for client-library access. Verified by live API calls with real agent credentials:

Identity	Own DB	Other domain DBs	shared-db	Cloud Run reach
power / cooling / facilities / cost	200	403	403	gateway only
orchestrator-agent-sa	n/a	403 (all four)	200	gateway only
gateway-agent-sa	none	none	none	the 4 specialists
web-bff-sa (public tier)	none	none	200 read-only	orchestrator only

The orchestrator row is the architectural claim: it is **incapable** of reading raw power, cooling, facilities or cost data. "The orchestrator only sees verdicts" is therefore a property of the platform, not a promise in a README. Equally deliberate is the last row — the only internet-facing service holds the least privilege in the system: read-only, one database, and no Vertex AI permission at all.

Beyond the checklist: harness engineering

The track asks whether an organisation can *trust* these agents. Most of that trust is not created by any of the seven tools — it comes from what surrounds the model call. In GridMind the model call is one step inside a harness, not the agent itself.

Property	Implementation and evidence
Structured I/O	Every call in and out is a schema-validated pydantic `AgentVerdict`. Enforced twice: Gemini's response schema constrains the shape, pydantic then validates it, because "matches the schema" and "is semantically valid" are different claims.
Scoped tool access	All Firestore reads pass through one module that can only open the database the agent's identity is bound to. The enforcement is IAM; the code is a fast, legible failure for an obvious mistake.
Retry with correction	Three attempts with exponential backoff. The violation text is fed back into the next attempt, so attempt two literally reads "your answer violated a hard constraint: zone-b has 561 kW of headroom against a 792 kW request". It usually self-corrects.
Fail safe, never open	If all retries are exhausted the agent returns *infeasible* plus an escalation flag. An agent that cannot reason reliably about a 90 MW electrical envelope stops the line rather than approving and hoping.
Guardrails on output	Every verdict is re-checked against the same ground truth the agent was given. Tested against five deliberately wrong verdicts — all five blocked, including one that passed its headline verdict while smuggling an impossible fallback into `proposed_alternative`.

A defect this audit found in its own system

Worth recording, because it is the kind of failure a compliance checklist misses. All 14 database-isolation checks passed while a single shared `notes` field had copied the sentence *"5 liquid-ready racks against 6 needed"* into all four domain databases. Every agent could therefore solve the problem alone, and the multi-agent premise had quietly collapsed — with the security tests still green.

Airtight permissions plus a leaky payload is not isolation. A second test now scans every field name and string value written to each database and fails the seed if one domain's facts appear in another's store. Both tests are required; neither is sufficient.

Closing the two gaps

Gap	Proposed work	Effort	Value
Agent Registry	A registry collection plus a discovery endpoint and a dashboard panel. Each agent self-registers on startup with its version, capability description, input/output schema, the single data domain it is scoped to, and its owning team. Directly answers "how does an organisation discover your agents".	~2 h	HIGH - closes the weakest area against this track
Model Armor	Screen inter-agent messages at the gateway, which is already the single choke point every specialist call passes through. The API is enabled-able in this project and the integration seam is documented in the gateway source.	~2 h	HIGH - the named tool for the security focus
Async Agent Runtime	Job-based negotiation: submit returns an id, the client polls. Removes the 60-90 s blocking request and moves closer to GEAP's long-running execution model.	~2 h	MEDIUM - better UX, weaker narrative gain

Recommendation: build the Registry and Model Armor. Together they take the scorecard from 4/7 to 6/7 and, more importantly, convert the two weakest answers on this track's own stated focus into demonstrable ones.

Reproducing every claim in this document

```
bash infra/iam/03_verify_isolation.sh   # 14 database isolation checks
bash scripts/demo_denial.sh         # all four security layers, live
python -m seed.leak_check            # cross-domain payload leak test
bash scripts/smoke_deployed.sh      # public surface + full negotiation
gcloud logging read 'jsonPayload.correlation_id="gm-..."'  # one decision's full reasoning chain
```